

EASTERN INTERNATIONAL UNIVERSITY

BECAMEX BUSINESS SCHOOL



MIS 443: Business Data Management

Name	IRN
Văn Vũ Quỳnh Như	2232300079

Quarter: 4/ 2025-2026

1. Student name and ID

- **Name:** Văn Vũ Quỳnh Như
- **IRN:** 2232300079
- **Course:** MIS443

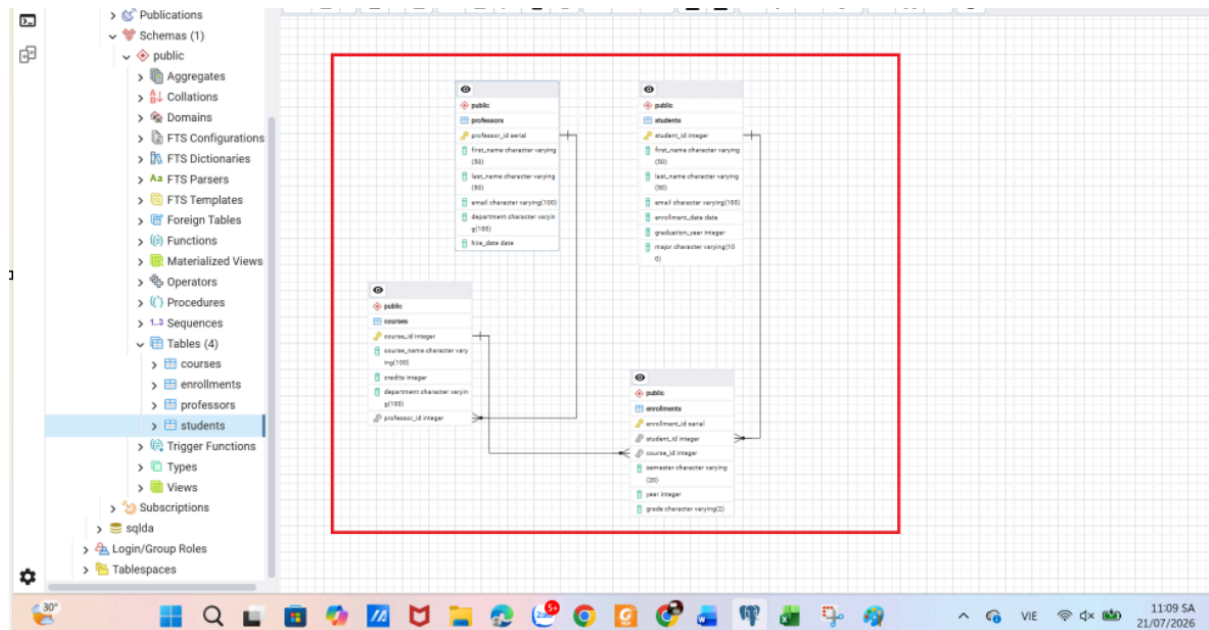
2. Introduction to the School database

The **school** database is a relational database schema sourced from the SQL Practice Online system. It is designed to store and manage the core operational data of a university or educational institution. This database simulates a real-world educational environment, encompassing the management of student records, professor profiles, course catalogs, and the tracking of student enrollments and academic performance (grades) across various semesters. For this project, the data was extracted into CSV files and implemented locally using PostgreSQL (via pgAdmin 4) to facilitate in-depth data analysis and querying.

3. Description of the main tables and relationships The School database consists of four interconnected main tables:

- **students Table:** Stores personal and academic information of the students. The `student_id` column serves as the Primary Key (PK). Other attributes include first and last name, email, enrollment date, expected graduation year, and major.
- **professors Table:** Contains profiles of the university's faculty members. The `professor_id` is the Primary Key (PK). This table tracks details such as name, email, associated department, and hire date.
- **courses Table:** Manages the catalog of available subjects. The table uses `course_id` as the Primary Key (PK). It includes a Foreign Key (FK), `professor_id`, which references the professors table. This establishes a 1-to-Many (1-N) relationship, meaning one professor can teach multiple courses, but each course is assigned to a specific professor.
- **enrollments Table:** Acts as a junction table to resolve the Many-to-Many (N-N) relationship between students and courses. Its Primary Key is `enrollment_id`. It utilizes two Foreign Keys: `student_id` (referencing the students table) and `course_id` (referencing the courses table). Additional data includes the semester, year, and the grade achieved by the student.

4. Entity–Relationship Diagram



5. Summary of the database implementation

The database implementation was executed systematically using pgAdmin 4. First, a new database named school_SQL Practice Online was initialized. Next, the table structures were built using DDL (Data Definition Language) commands (CREATE TABLE), carefully defining the appropriate data types such as VARCHAR for text strings, INT for whole numbers, and DATE for time-related fields. To ensure strict adherence to Referential Integrity Constraints, a logical creation and insertion order was followed. Independent tables without foreign keys (students and professors) were created and populated first. Subsequently, the dependent tables containing foreign keys (courses and enrollments) were created and populated. The raw data was imported from the four provided CSV files using pgAdmin's Import/Export utility.

6. Summary of the completed SQL questions This project successfully addressed 30 practical SQL questions related to the School schema. These queries covered essential data analysis techniques in SQL, including:

- Basic data retrieval and filtering using SELECT and WHERE clauses.
- Combining cross-table data for relational analysis using INNER JOIN and LEFT JOIN.
- Summarizing academic metrics using Aggregation functions (COUNT, AVG, MAX) combined with GROUP BY and HAVING clauses.
- Formatting query outputs using ORDER BY for logical sorting.
- Handling complex data filtering requirements utilizing Subqueries and Common Table Expressions (CTEs).

7. Selected pgAdmin screenshots

Q1: Return the complete student roster from the Students table.

The screenshot shows a PostgreSQL client interface with the following components:

- Query Editor:** Contains the SQL query: `-- Q1: Return the complete student roster from the students table.`
`SELECT * FROM students;`
- Data Output:** Displays the results of the query in a table format. The table has 7 columns: `student_id` (integer), `first_name` (character varying (50)), `last_name` (character varying (50)), `email` (character varying (100)), `enrollment_date` (date), `graduation_year` (integer), and `major` (character varying (100)). The results show 7 rows of student data.

student_id	first_name	last_name	email	enrollment_date	graduation_year	major	
1	1001	Alex	Thompson	alex.thompson@university.edu	2022-09-01	2026	Computer Science
2	1002	Maria	Rodriguez	maria.rodriguez@university.edu	2022-09-01	2026	Mathematics
3	1003	James	Wilson	james.wilson@university.edu	2021-09-01	2025	Physics
4	1004	Emily	Chen	emily.chen@university.edu	2023-09-01	2027	Computer Science
5	1005	Robert	Johnson	robert.johnson@university.edu	2022-09-01	2026	Chemistry
6	1006	David	Smith	david.smith@university.edu	2023-09-01	2027	Computer Science
7	1007	Sarah	Lee	sarah.lee@university.edu	2022-09-01	2026	Mathematics

Q2: Return students who are majoring in Computer Science.

The screenshot shows a PostgreSQL client interface with the following components:

- Query Editor:** Contains the SQL query: `-- Q1: Return the complete student roster from the students table.`
`SELECT * FROM students;`
`-- Q2: Return students who are majoring in Computer Science.`
`SELECT first_name, last_name, graduation_year`
`FROM students`
`WHERE major LIKE '%Computer Science';`
- Data Output:** Displays the results of the query in a table format. The table has 3 columns: `first_name` (character varying (50)), `last_name` (character varying (50)), and `graduation_year` (integer). The results show 2 rows of student data.

first_name	last_name	graduation_year
Alex	Thompson	2026
Emily	Chen	2027

Q3: Return all courses ordered by credit hours from highest to lowest.

Query Editor Interface showing SQL queries and results.

Query 3: Return all courses ordered by credit hours from highest to lowest.

```

SELECT course_name, credits
FROM courses
ORDER BY credits DESC;

```

Data Output:

course_name	credits
Data Structures	4
Calculus I	4
General Physics I	4
General Chemistry	4
Introduction to Programm...	3

Successfully run. Total query runtime: 269 msec. 6 rows affected.

Q4: Return students who are expected to graduate in 2026.

Query Editor Interface showing SQL queries and results.

Query 4: Return students who are expected to graduate in 2026.

```

SELECT first_name, last_name, major
FROM students
WHERE graduation_year = 2026;

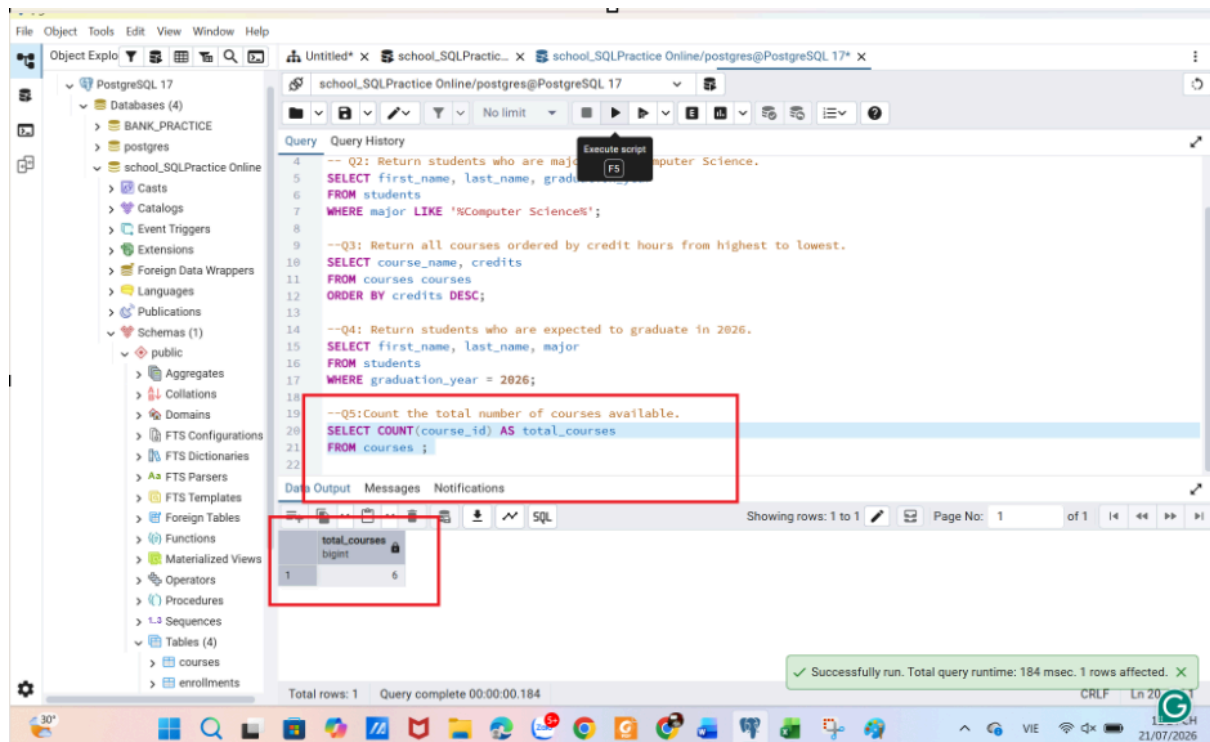
```

Data Output:

first_name	last_name	major
Alex	Thompson	Computer Science
Maria	Rodriguez	Mathematics
Robert	Johnson	Chemistry

Successfully run. Total query runtime: 161 msec. 3 rows affected.

Q5: Count the total number of courses available.



7. Conclusion

Through this project, a fully functional relational database was successfully reconstructed using PostgreSQL. The assignment reinforced practical skills in database design, establishing relationships, ensuring data integrity through constraints, and writing complex SQL queries for data analysis. These competencies are highly applicable to real-world business analytics and data engineering tasks.

8. Challenges encountered and solutions

- **Challenge 1: Foreign Key Violation during Data Import.**
 - *Description:* When attempting to import data from the CSV into the enrollments table, the system threw an error because the respective student_id and course_id values did not yet exist in the database.
 - *Solution:* Recognizing the data dependency issue, I adjusted the execution script and import sequence. I ensured that all records for the students and courses tables were fully imported first before initiating the import for the SCHOOL_enrollments.csv file.
- **Challenge 2: Handling Date Formats.**
 - *Description:* The date formats in the CSV files (e.g., enrollment_date) were formatted as DD/MM/YYYY, whereas PostgreSQL often defaults to the ISO format (YYYY-MM-DD).
 - *Solution:* I resolved this by correctly mapping the columns in the pgAdmin Import/Export dialog box and utilizing the appropriate DateStyle settings so the system parsed the strings correctly without throwing syntax errors.

9. Conclusion

This project successfully simulated the complete lifecycle of constructing a relational database system: from interpreting a schema and designing the architecture with proper constraints, to importing real-world data from flat files and executing analytical queries. Completing the 30 SQL challenges using PostgreSQL and pgAdmin 4 not only reinforced structural data thinking but also significantly improved data extraction and manipulation skills, providing a solid foundation for future work in Business Analytics.

10. Personal GitHub repository link

- **Repository Link:** <https://github.com/vvqnhu204>